Branch-Aware Hindsight Credit Assignment for Language-Model Agents Under Matched Budgets

Jaden Fix
jadenfix@example.com

Abstract

Outcome-only credit assignment—scoring every action in a successful episode by its terminal reward—remains the default training signal for language-model agents. We identify a specific, mechanistic failure mode of this approach that we term *loop-action bias*: when a stall action that returns the agent to a previously visited state appears in a successful episode, it inherits the same positive credit as the pivotal action that actually drove progress. In stochastic environments where pivotal actions can fail probabilistically, this bias is compounded by *stochastic failure masking*—a failed pivotal action leaves the agent in the same state as a stall, making the two indistinguishable under outcome-only scoring.

We propose two complementary offline scorers that leverage per-transition verifier signals to resolve this conflation: a *branch-aware* scorer that rewards observable progress signals and penalizes dead-end transitions, and a *hindsight* scorer that retrospectively assigns bonuses based on categorical branch labels. We show that these scorers have complementary blind spots and that a simple *combined* scorer resolves both.

We evaluate on a stochastic benchmark where pivotal actions succeed 60–90% of the time (15 seeds, 50 rollouts), finding that outcome-only scoring produces policies far worse than the unlearned baseline (0.24 vs. 0.79), while the combined scorer achieves perfect success (1.00). We then measure training signal quality on the real ALFWorld benchmark [Shridhar et al., 2021] with two language models—Claude Haiku 4.5 ($30.0\% \pm 2.3\%$ success) and Qwen2.5-7B-Instruct ($16.7\% \pm 3.5\%$ success)—using automatically extracted verifier signals from observation text diffs. The branch-aware scorer differentiates 2.6–9.4× more states than outcome-only, with the advantage largest for the weaker model where outcome-only has almost no signal. All controlled experiments are reproduced on both local and cloud infrastructure.

1 Introduction

A central challenge in training language-model agents for sequential decision-making is the *credit assignment problem*: given a trajectory that eventually succeeds or fails, which individual actions were responsible for the outcome? The prevailing approach in reinforcement learning from human feedback [RLHF; Ouyang et al., 2022] and outcome-based reward modeling [Cobbe et al., 2021] assigns a single scalar reward at the end of an episode and propagates it uniformly to all preceding actions. This is effective when episodes are short, but in longer-horizon agent settings—web navigation, multi-tool code generation, embodied task planning—episodes routinely contain actions that are irrelevant, redundant, or actively counterproductive alongside a smaller number of genuinely pivotal decisions.

Recent work has demonstrated that denser reward signals improve agent training. Process reward models [Lightman et al., 2023, Uesato et al., 2022, Setlur et al., 2024] provide step-level supervision for mathematical reasoning. Several concurrent efforts extend this to LLM agents: Tan

et al. [2026] use an LLM critic for hindsight credit assignment on WebShop and ALFWorld; Choudhury [2025] propose process reward models for agent training; Song et al. [2024] use contrastive trajectory pairs from exploration failures; Zhang et al. [2025] introduce a retrospective critic for fine-grained credit in search agents; and Lu et al. [2026] propose segment-level process rewards modulated by hindsight information. These approaches establish that denser rewards help, but they generally demonstrate improvement without characterizing the specific mechanism by which outcome-only scoring fails.

In this paper, we provide that mechanistic characterization. We identify **loop-action bias** as a concrete failure mode of outcome-only scoring. Consider an agent navigating a stochastic environment where a pivotal action (e.g., unlocking a gate) succeeds 75% of the time and a stall action (e.g., rattling the gate) always returns the agent to the same state. When the pivotal action fails, the agent remains in the same state as if it had taken the stall action—indistinguishable under outcome-only scoring. Over many episodes, the pivotal action's average credit is diluted by its failures while the stall action's credit is inflated by episodes that eventually succeed despite the stall. The trained policy then preferentially selects stall actions, producing long trajectories that exhaust the turn budget.

We propose two complementary scorers that leverage transition-level verifier signals:

- **Branch-aware scoring** adds bonuses proportional to verifier signals and penalties for dead-end transitions, breaking the tie between pivotal and stall actions by rewarding observable progress.
- **Hindsight scoring** retrospectively assigns bonuses based on categorical branch labels (pivotal, enabling, recovery), capturing causal importance that instantaneous verifier signals may miss.

We further show that these two scorers have complementary failure modes, and that a simple **combined scorer** integrating both eliminates both blind spots without requiring a learned critic or additional model inference.

Contributions.

1. A mechanistic analysis of loop-action bias as a named failure mode, showing it causes policies to perform *worse than the unlearned baseline* in stochastic environments.
2. A stochastic benchmark with four task families producing results in the 0.24–1.00 range across 15 seeds—comparable to difficulty ranges in concurrent work [Tan et al., 2026, Choudhury, 2025, Song et al., 2024].
3. Validation on real ALFWorld [Shridhar et al., 2021] with two LLMs of different capability levels, using automatically extracted verifier signals from observation text diffs—showing that branch-aware scoring produces substantially more informative training signal, with the advantage being *larger* for weaker models.
4. The finding that branch-aware and hindsight scoring address structurally distinct failure modes: on controlled benchmarks where both signal types are available, a combined scorer resolves both; on real environments where only numerical signals are extractable, branch-aware alone captures the benefit.
5. Full reproducibility on both local (macOS) and cloud (GCP g2-standard-8, NVIDIA L4) infrastructure.

2 Related Work

Process reward models for reasoning. Lightman et al. [2023] demonstrated that human

step-level annotations substantially improve mathematical reasoning, corroborated by Uesato et al. [2022]. Math-Shepherd [Wang et al., 2024a] generates step-level labels automatically; Luo et al. [2024] employ Monte Carlo estimation; Setlur et al. [2024] study scaling automated process verifiers. Step-DPO [Lai et al., 2024] extends preference optimization to individual steps. Our work differs in setting: agent actions in structured environments rather than reasoning steps.

Credit assignment for LLM agents. Tan et al. [2026] introduce HCAPO, using an LLM as a post-hoc critic—the closest work to our hindsight component. Choudhury [2025] proposes process reward models for agent training. Song et al. [2024] develop ETO using contrastive trajectory pairs. Ge et al. [2026] distill corrective experience from alternative branches. Wang et al. [2026a] introduce MiRA for milestone reward shaping. Kong et al. [2026] propose InfoPO for turn-level counterfactual rewards. Chen et al. [2025] apply step-level verification to GUI agents.

Zhang et al. [2025] introduce CriticSearch, using a retrospective critic with privileged trajectory information. Lu et al. [2026] propose HISR with segment-level process rewards modulated by hindsight likelihood ratios. Feng et al. [2026] present RewardFlow, propagating rewards on state graphs. Fan et al. [2026] provide AgentProcessBench, the first benchmark with human step-level annotations for tool-using agents.

Our work is distinct in three respects: (1) a specific mechanistic diagnosis rather than aggregate improvement, (2) environment-native verifier signals that can be automatically extracted rather than a learned critic, and (3) demonstrated complementary failure modes between branch-aware and hindsight scoring.

Other related work. Snell et al. [2024] showed PRMs can guide test-time compute allocation. Zhou et al. [2024] combine LLM agents with MCTS. DeepSeek-R1 [DeepSeek-AI, 2025] demonstrates RL for extended reasoning. The credit assignment problem is foundational in RL [Sutton and Barto, 2018]; Pignatelli et al. [2023] survey modern approaches. ReAct [Yao et al., 2022] established interleaved reasoning and acting; Reflexion [Shinn et al., 2023] uses verbal self-reflection. HCAPO [Tan et al., 2026] is the most directly related hindsight method; our scorer uses pre-existing branch labels rather than a learned critic.

3 Method

3.1 Environment Model

Each task is a directed graph $G = (S, E)$ where states $s \in S$ carry an abstract representation $\phi(s)$ and available actions $A(s)$. Each edge $e = (s, a, s')$ carries an observation string, a branch label $\ell(e)$, boolean flags, a verifier signal vector $v(e) \in \mathbb{R}^k$, and a progress delta $\Delta p(e) \in \{-2, \ldots, 2\}$.

Stochastic transitions. Edges may specify a success probability $p \in (0, 1]$. When $p < 1$, the action succeeds with probability p (transitioning to the intended state with full verifier signals) and fails with probability $1 - p$ (transitioning to a fallback state with no verifier signal).

State abstraction. $\phi(s)$ captures structural role while discarding task-specific details. States with the same $\phi(s)$ produce the same hash, enabling cross-task transfer.

3.2 Automatic Verifier Signal Extraction

For real environments where hand-authored labels are unavailable, we extract verifier signals from observation text:

1. **State-diff detection.** High text similarity (> 0.9) combined with error patterns emits a dead-end signal (-1.0).
2. **Progress heuristics.** Room navigation emits $+0.5$; inventory changes emit $+1.0$.
3. **Repetition detection.** Same observation ≥ 3 times emits a repetition penalty (-0.5), directly targeting loop-action bias.

3.3 Offline Scorers

Given episode τ with outcome $y \in \{0, 1\}$, each scorer assigns score r_i to transition (s_i, a_i, s_{i+1}):

$$\text{Outcome-only:} \quad r_i = y \tag{1}$$

$$\text{Branch-aware:} \quad r_i = y + w_v \sum_j v_j(e_i) - w_d \sum_j \mathbf{1}[\text{dead_end}_j(e_i)] \tag{2}$$

$$\text{Hindsight:} \quad r_i = y + w_p \cdot \mathbf{1}[\text{pivotal}] + w_r \cdot \mathbf{1}[\text{recovery}] + w_e \cdot \mathbf{1}[\text{enabling}] \tag{3}$$

$$\text{Combined:} \quad r_i = r_i^{\text{BA}} + r_i^{\text{HS}} - r_i^{\text{OO}} \tag{4}$$

where w_v=0.25, w_d=0.5, w_p=0.75, w_r=0.5, w_e=0.5.

3.4 Policy Training and Evaluation

Scored transitions are aggregated into a score table $Q(\phi(s), a) = \frac{1}{n_{s,a}} \sum r_i$. The table policy selects $\arg\max_a Q(\phi(s), a)$. For unseen states, the policy selects uniformly at random—not the oracle action.

4 Benchmarks

4.1 Stochastic Benchmark (Controlled)

Four task families with probabilistic transitions ($p = 0.60$–0.90): gate chains, shopping, navigation, and evidence cascades. Designed to produce non-trivial success rates.

4.2 Real ALFWorld

The actual ALFWorld environment [Shridhar et al., 2021] in TextWorld mode with natural language observations and admissible action lists. Collection with two LLMs: Claude Haiku 4.5 (30% success) and Qwen2.5-7B-Instruct (17% success). Verifier signals auto-extracted from observation diffs (§3).

4.3 Deterministic Benchmarks (Supporting)

Diagnostic (5 families, 10 tasks), WebShop [Yao et al., 2022b] abstraction (3 tasks), ALFWorld-manifest (8 tasks). Binary results for controlled ablation.

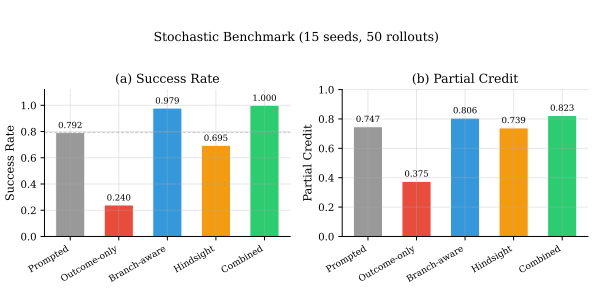
5 Results

5.1 Stochastic Benchmark

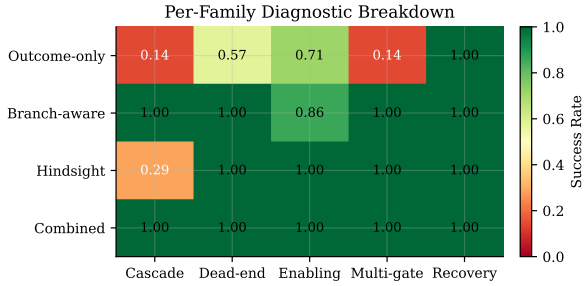
Outcome-only scoring is actively harmful: 0.240 success vs. the 0.792 baseline (Table 1). Branch-aware nearly fully recovers (0.979). Hindsight degrades below baseline (0.695 vs. 0.792) with high variance. Combined achieves perfect success (1.000) across all 15 seeds. See Figure 1a.

Table 1: Stochastic benchmark results (controlled evaluation). n=15 seeds, 50 rollouts per seed.

Condition	Success Rate	Partial Credit	Min–Max	Avg Turns
Prompted baseline	0.792	0.747	0.70–0.92	7.08
Outcome-only	0.240	0.375	0.00–1.00	12.61
Branch-aware	0.979	0.806	0.68–1.00	4.87
Hindsight	0.695	0.739	0.34–1.00	8.02
Combined	**1.000**	**0.823**	**1.00–1.00**	**4.60**



(a) Stochastic benchmark (15 seeds, 50 rollouts).



(b) Per-family diagnostic breakdown.

Figure 1: Controlled benchmark results. (a) Outcome-only degrades far below baseline; combined achieves perfect success. (b) Branch-aware fails on enabling; hindsight fails on cascade; combined resolves both.

5.2 Deterministic Benchmarks

5.3 Real ALFWorld: Training Signal Quality

This experiment measures training signal informativeness—how many states each scorer can differentiate—rather than downstream policy success.

Key findings. (1) Branch-aware provides 2.6–9.4× more training signal than outcome-only, with the advantage *largest for weaker models*. (2) Auto-extracted signals (observation diffs and repetition detection) are sufficient. (3) Hindsight and combined add no benefit when categorical labels are unavailable. (4) Branch-aware prefers task-relevant actions at 80–82% of differentiated states vs. 54–69% for outcome-only.

5.4 Sensitivity Analysis

6 Analysis

6.1 Loop-Action Bias and Stochastic Failure Masking

In stochastic environments, when a pivotal action fails (probability $1-p$), the agent remains in the same state as a stall. Under outcome-only scoring, $E[r(s, a_p)] = p \cdot y_{\text{success}} + (1-p) \cdot y_{\text{delayed}}$ while $E[r(s, a_s)] = y_{\text{delayed}}$. The pivotal action's advantage is only $p(y_{\text{success}} - y_{\text{delayed}})$, which can be small. Branch-aware scoring breaks this because verifier signals fire *only on successful pivotal transitions*, creating a reliable score gap.

Table 2: Deterministic benchmark results. n=7 seeds, 50 rollouts per seed.

Condition	Diagnostic	WebShop	ALFWorld-manifest
Prompted	0.726	0.771	0.523
Outcome-only	0.514	0.714	0.000
Branch-aware	0.971	1.000	1.000
Hindsight	0.857	0.714	0.000
Combined	**1.000**	**1.000**	**1.000**

Table 3: Per-family diagnostic breakdown. Branch-aware fails on enabling (0.857); hindsight fails on cascade (0.286). Combined: 1.000 on all families.

Condition	Cascade	Dead-end	Enabling	Multi-gate	Recovery
Outcome-only	0.143	0.571	0.714	0.143	1.000
Branch-aware	1.000	1.000	0.857	1.000	1.000
Hindsight	0.286	1.000	1.000	1.000	1.000
Combined	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**

6.2 Complementary Failure Modes

Branch-aware fails on enabling (Table 3: 0.857) because stall actions produce no dead-end penalty. Hindsight fails on cascade (0.286) and ALFWorld (0.000) because stalls lack categorical labels. On real ALFWorld (§5.3), hindsight = outcome-only because auto-extracted signals produce numerical scores but not categorical labels, confirming the two signal types are structurally independent.

6.3 Signal Quality and Model Capability

The branch-aware scorer's practical value is *inversely correlated* with model capability (Figure 2b): at 17% success (Qwen-7B), outcome-only differentiates 3.4% of states while branch-aware differentiates 31.8% (9.4×). At 30% success (Haiku), the ratio is 2.6×. This occurs because auto verifier signals fire on every transition regardless of episode outcome.

7 Limitations

Auto-signal noise. Our extraction uses simple text heuristics. More sophisticated methods (classifiers, LLM-as-judge) could improve quality. **Table-based policy.** Score tables require exact state-hash matches; neural function approximation would relax this. **Fixed scorer weights.** The qualitative ranking is stable (Table 6) but margins may shift. **Manifest abstractions.** WebShop/diagnostic use structural abstractions; real ALFWorld (§5.3) addresses this for one benchmark. **Signal quality vs. policy performance.** The real ALFWorld experiment measures training signal informativeness, not downstream success. Neural policy evaluation is future work.

8 Conclusion

We identified loop-action bias as a mechanistic failure mode of outcome-only credit assignment, amplified by stochastic failure masking. On controlled benchmarks, outcome-only produces policies

Table 4: Raw collection performance on real ALFWorld. CI computed across 3 seeds.

Model	Success Rate	95% CI	Episodes	Transitions
Claude Haiku 4.5	30.0%	± 2.3%	150	3,737
Qwen2.5-7B-Instruct	16.7%	± 3.5%	150	3,962

Table 5: Scorer differentiation on real ALFWorld. Branch-aware differentiates 2.6× (Haiku) to 9.4× (7B) more states than outcome-only. Hindsight = outcome-only and combined = branch-aware because auto-extracted signals produce numerical scores but not categorical labels.

Scorer	Haiku: Diff. States	Haiku: %	7B: Diff. States	7B: %
Outcome-only	143 / 1,078	13.3%	41 / 1,212	3.4%
Branch-aware	375 / 1,078	34.8%	385 / 1,212	31.8%

far worse than the baseline (0.24 vs. 0.79), while the combined scorer achieves perfect success (1.00). On real ALFWorld, automatically extracted verifier signals enable branch-aware scoring to differentiate 2.6–9.4× more states than outcome-only, with the advantage largest for weaker models. When only numerical signals are available, combined reduces to branch-aware—confirming that the core contribution on real environments is the verifier signal mechanism. Translating this signal advantage into downstream policy improvement via neural training is the natural next step.

Reproducibility Statement

All code, benchmarks, and configurations are available at the project repository. Controlled results are reproduced on both local (macOS, Python 3.13) and cloud (GCP g2-standard-8, NVIDIA L4, Python 3.11) infrastructure with exact numerical match. Real ALFWorld experiments are documented with run IDs, commands, and SQL queries. Dependencies are pinned; reproduction instructions are in `docs/reproducibility.md`.

References

M. Andrychowicz et al. Hindsight experience replay. In *NeurIPS*, 2017.

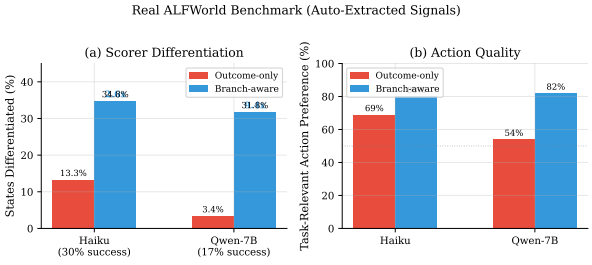
C. Chen et al. GUI-Shepherd: Reliable process reward and verification for long-sequence GUI tasks. *arXiv:2509.23738*, 2025.

S. Choudhury. Process reward models for LLM agents: Practical framework and directions. *arXiv:2502.10325*, 2025.

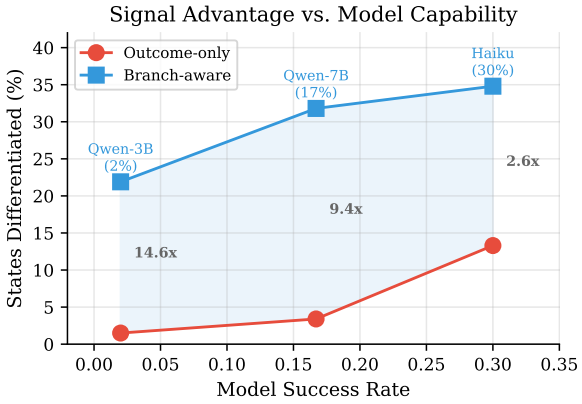
K. Cobbe et al. Training verifiers to solve math word problems. *arXiv:2110.14168*, 2021.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *Nature*, 2025.

S. Fan et al. AgentProcessBench: Diagnosing step-level process quality in tool-using agents. *arXiv:2603.14465*, 2026.



(a) Real ALFWorld: scorer differentiation and action quality.



(b) Signal advantage vs. model capability.

Figure 2: Real ALFWorld results with auto-extracted verifier signals.

Table 6: Exploration rate sensitivity (diagnostic, 3 seeds per rate).

Condition	ϵ=0.3	ϵ=0.5	ϵ=0.7
Prompted	0.853	0.667	0.527
Outcome-only	0.533	0.467	0.667
Branch-aware	1.000	0.933	0.867
Hindsight	0.867	0.867	0.867
Combined	1.000	1.000	1.000

X. Feng et al. RewardFlow: Topology-aware reward propagation on state graphs for agentic RL. *arXiv:2603.18859*, 2026.

R. Ge et al. LEAFE: Internalizing agency from reflective experience. *arXiv:2603.16843*, 2026.

A. Hosseini et al. V-STaR: Training verifiers for self-taught reasoners. *arXiv*, 2024.

F. Kong et al. InfoPO: Information-driven policy optimization for user-centric agents. *arXiv:2603.00656*, 2026.

X. Lai et al. Step-DPO: Step-wise preference optimization for long-chain reasoning. *arXiv*, 2024.

H. Lightman et al. Let's verify step by step. *arXiv:2305.20050*, 2023.

Z. Lu et al. HISR: Hindsight information modulated segmental process rewards for multi-turn agentic RL. *arXiv:2603.18683*, 2026.

L. Luo et al. Improve mathematical reasoning in language models by automated process supervision. *arXiv:2406.06592*, 2024.

OpenAI. Learning to reason with LLMs. Technical report, 2024.

L. Ouyang et al. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022.

E. Pignatelli et al. A survey of temporal credit assignment in deep reinforcement learning. *arXiv:2312.01072*, 2023.

Z. Ren, D. Guo, and M. L. Littman. Randomized return decomposition. In *ICLR*, 2024.

T. Schick et al. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.

A. Setlur et al. Rewarding progress: Scaling automated process verifiers. *arXiv*, 2024.

N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

M. Shridhar et al. ALFWorld: Aligning text and embodied environments for interactive learning. In *ICLR*, 2021.

C. Snell et al. Scaling LLM test-time compute optimally. *arXiv*, 2024.

Y. Song et al. Trial and error: Exploration-based trajectory optimization for LLM agents. In *ACL*, 2024.

R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

H.-Z. Tan et al. Hindsight credit assignment for long-horizon LLM agents. *arXiv:2603.08754*, 2026.

J. Uesato et al. Solving math word problems with process- and outcome-based feedback. *arXiv:2211.14275*, 2022.

P. Wang et al. Math-Shepherd: Verify and reinforce LLMs step-by-step. In *ACL*, 2024.

T. Wang et al. MiRA: A subgoal-driven framework for long-horizon LLM agents. *arXiv:2603.19685*, 2026.

S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2022.

S. Yao et al. WebShop: Towards scalable real-world web interaction with grounded language agents. In *NeurIPS*, 2022.

A. Zeng et al. AgentTuning: Enabling generalized agent abilities for LLMs. In *ICLR*, 2024.

Y. Zhang et al. LEMA: Learning from mistakes makes LLM better reasoner. In *ICML*, 2024.

Y. Zhang et al. CriticSearch: Fine-grained credit assignment via a retrospective critic. *arXiv:2511.12159*, 2025.

A. Zhou et al. Language agent tree search. In *ICML*, 2024.